

Formalizing Finite-Class Learning Guarantees in Lean 4 with an MA-LoT-Inspired Multi-LLM Proof Repair Workflow

Dhruv

Abstract

This project formalizes finite hypothesis class learning guarantees in Lean 4 and studies an MA-LoT-inspired workflow for repairing AI-generated Lean proofs. The mathematical topic is empirical risk minimization for finite hypothesis classes in the realizable and agnostic settings. The strongest fully verified Lean result is the deterministic agnostic ERM theorem: under uniform convergence, ERM competes with a comparator hypothesis up to an additive 2ϵ term. The realizable and high-probability agnostic guarantees are represented using explicit assumed concentration results rather than by formalizing probability theory from scratch.

The workflow contribution documents how Codex and Claude generate proof attempts from a shared theorem prompt, how Lean verifier feedback exposes errors, how repair iterations fix those errors, and how comparison across models produces both human-facing explanations and model-facing repair instructions.

Contents

1	Reader's Guide	3
2	Main Result Summary	3
3	Assumptions and Scope at a Glance	4
3.1	Mathematical Assumptions	4
3.2	Assumptions Represented Explicitly in Lean	4
3.3	Implementation Scope Choices	4
4	Project Overview	5
5	Motivation: Why Assumptions Matter	5
6	Mathematical Setup	5
6.1	Binary Classification	5
6.2	True Risk and Empirical Risk	6
7	Realizable Setting	6
7.1	Assumption	6
7.2	Consistency of ERM	6
7.3	Bad Hypotheses	6
7.4	One Bad Hypothesis	7
7.5	Union Bound	7
7.6	Sample Complexity	7
7.7	Lean Status	8

8	Agnostic Setting	8
8.1	Assumption Removed	8
8.2	Uniform Convergence	8
8.3	Deterministic Agnostic ERM Lemma	8
8.4	Lean Status	9
8.5	High-Probability Agnostic Bound	10
9	Why the Rates Differ	10
10	Lean Formalization	11
10.1	Design Choice: Abstract Risk	11
10.2	Main Definitions	11
10.3	Lean Theorem Interfaces and Assumptions	11
10.4	Formalization Boundary	13
11	Repository Layout	13
11.1	Lean Files	13
11.2	Notes and Proof Explanation	14
11.3	Presentation Materials	14
12	MA-LoT-Inspired Proof Repair Workflow	14
12.1	What MA-LoT Contributes Conceptually	14
12.2	Adapted Workflow	14
12.3	Why This Matters	15
12.4	AI Collaboration Reflection	15
13	Multi-Model Comparison Workflow	16
13.1	Blind Generation	16
13.2	Generated Proof Artifacts	16
13.3	Comparison Script	16
13.4	Repair and Simplification	17
13.5	How Comparison Prevents Shortcuts	17
14	Proof Repair Case Study	17
14.1	Failure Pattern 1: Uniform Convergence Treated Too Informally	18
14.2	Failure Pattern 2: ERM Used Before Matching the Goal	18
14.3	Failure Pattern 3: Arithmetic Hidden by “Rearranging”	18
15	Verification and Reproducibility	18
15.1	Build Command	18
15.2	Comparison Command	18
16	Limitations	19
17	Future Work	19
18	Conclusion	19

A	Appendix A: File Index	20
A.1	Core Lean Formalization	20
A.2	Mathematical Notes	20
A.3	MA-LoT-Inspired Workflow	20
A.4	Multi-Model Workflow	20
A.5	Proof Refactoring	21
A.6	Presentation and Explainability	21
A.7	Final Submission Files	21
B	Appendix B: Main Lean Theorem Names	22
C	Appendix C: Final Submission Checklist	22

1 Reader’s Guide

This report is organized around the three requirements in the final project instructions.

1. **Formalized proof.** The Lean code is in `LearningTheoryProject/`. The main fully proved theorem is `agnostic_erm_deterministic` in `LearningTheoryProject/Agnostic.lean`. Probability concentration results are stated explicitly as assumptions in `LearningTheoryProject/AssumedResults.lean`.
2. **Designed proof presentation.** The mathematical proof is explained in this report, especially in the realizable and agnostic proof sections. Additional reader-facing materials are in `notes/` and `presentation/`, including an annotated proof walkthrough, dependency graph, paper-versus-Lean table, and final slide deck.
3. **AI workflow.** The MA-LoT-inspired workflow is documented in `ai_workflow/`, `malot_workflow/`, `multi_model_workflow/`, `generated_proofs/`, and `proof_refactoring/`. These files record prompts, generated proof attempts, Lean verifier feedback, repair iterations, comparison reports, and reflection.

The most important formalization boundary is this: the deterministic agnostic ERM theorem is fully proved in Lean, while probability concentration results such as Hoeffding-style uniform convergence are stated as explicit assumptions.

2 Main Result Summary

Result	Mathematical meaning	Lean status
Realizable guarantee	Consistent hypotheses have small true risk with high probability.	Wrapper theorem using an assumed concentration result.
Agnostic deterministic theorem	Under uniform convergence, ERM is within 2ϵ of a comparator.	Fully proved in Lean.
Agnostic high-probability guarantee	ERM is close to the best hypothesis in the class with high probability.	Uses assumed uniform convergence plus the verified deterministic theorem.

3 Assumptions and Scope at a Glance

The project deliberately separates mathematical assumptions, Lean assumptions, and implementation choices.

3.1 Mathematical Assumptions

- The hypothesis class $\mathcal{H}$ is finite.
- Training examples are drawn independently from an unknown distribution $\mathcal{D}$.
- In the realizable setting, there exists $h^* \in \mathcal{H}$ such that $L_{\mathcal{D}}(h^*) = 0$.
- In the agnostic setting, no perfect classifier is assumed. The learner is compared with a best-in-class hypothesis h^* .
- The agnostic deterministic theorem assumes uniform convergence: empirical risk and true risk are within ϵ for every hypothesis in the class.

3.2 Assumptions Represented Explicitly in Lean

The file `LearningTheoryProject/AssumedResults.lean` contains the probability-facing assumptions. The key assumed interfaces are:

- `HighProbabilityEvent`: an abstract marker for “with probability at least $1 - \delta$.”
- `highProbability_mono`: an assumed monotonicity principle for high-probability events.
- `finiteClass_realizable_concentration`: the assumed finite-class concentration result for the realizable setting.
- `finiteClass_agnostic_uniform_convergence`: the assumed finite-class uniform convergence result for the agnostic setting.

3.3 Implementation Scope Choices

- Risks are represented as abstract `Nat`-valued quantities in Lean. This keeps the deterministic order proof lightweight.
- The usual logarithmic sample complexity rates are proved and explained mathematically in this report, but the Lean code represents the sample-size requirements through abstract sample-condition predicates.
- The project does not formalize measure theory, probability spaces, Hoeffding’s inequality, or the finite-class union bound from first principles.
- The project is MA-LoT-inspired. It does not implement the full MA-LoT system.

4 Project Overview

The project has two connected goals.

1. **Lean formalization goal.** Formalize the structure of finite hypothesis class learning guarantees in Lean 4. The project covers the realizable setting and the agnostic setting, while keeping probability-heavy concentration facts explicit as assumptions.
2. **Proof workflow goal.** Study an MA-LoT-inspired proof repair workflow in which AI-generated Lean proofs are checked by Lean, repaired from verifier feedback, compared across models, simplified only after verification, and translated into human-readable explanations.

The central communication theme is:

Verification is not the same as understanding.

Lean can verify that a proof term is correct. It does not automatically explain why the theorem matters, what informal assumptions were hidden in the paper proof, or why an AI-generated proof failed. The repository therefore includes both machine-checkable Lean code and human-readable proof artifacts.

5 Motivation: Why Assumptions Matter

No Free Lunch-style results say that learning is impossible without assumptions. If every possible labeling function is allowed, then no learning algorithm is universally better than another. This does not contradict finite-class learning guarantees. Instead, it explains why assumptions are necessary.

This project studies a classical positive result after adding a concrete assumption:

The hypothesis class $\mathcal{H}$ is finite.

Finiteness makes learning possible in the standard proof because it allows a union bound over all hypotheses. In the realizable case, the proof rules out bad hypotheses that nevertheless fit the sample. In the agnostic case, the proof uses uniform convergence to relate empirical risk to true risk for every hypothesis in the class.

6 Mathematical Setup

6.1 Binary Classification

Let $\mathcal{X}$ be an input space and let $\mathcal{Y} = \{0, 1\}$ be the label space. A hypothesis is a classifier

$$h : \mathcal{X} \rightarrow \mathcal{Y}.$$

The hypothesis class is finite:

$$\mathcal{H} = \{h_1, h_2, \dots, h_N\}.$$

A data point is

$$z = (x, y),$$

where $x \in \mathcal{X}$ and $y \in \mathcal{Y}$. The data distribution is an unknown distribution $\mathcal{D}$ over $\mathcal{X} \times \mathcal{Y}$.

A training sample is

$$S = (z_1, z_2, \dots, z_m),$$

where each $z_i = (x_i, y_i)$ is drawn independently from $\mathcal{D}$.

6.2 True Risk and Empirical Risk

Definition 1 (True risk). For a hypothesis h , the true risk is

$$L_{\mathcal{D}}(h) = \Pr_{(x,y) \sim \mathcal{D}}[h(x) \neq y].$$

This is the error probability on a fresh random example from the data distribution.

Definition 2 (Empirical risk). For a hypothesis h and sample S , the empirical risk is

$$L_S(h) = \frac{1}{m} \sum_{i=1}^m \mathbf{1}[h(x_i) \neq y_i].$$

This is the fraction of training examples that h classifies incorrectly.

Definition 3 (Empirical risk minimization). An empirical risk minimizer is a hypothesis

$$\hat{h} \in \arg \min_{h \in \mathcal{H}} L_S(h).$$

Equivalently,

$$\forall h \in \mathcal{H}, \quad L_S(\hat{h}) \leq L_S(h).$$

7 Realizable Setting

7.1 Assumption

The realizable setting assumes that the hypothesis class contains a perfect classifier:

$$\exists h^* \in \mathcal{H} \quad \text{such that} \quad L_{\mathcal{D}}(h^*) = 0.$$

In words, at least one hypothesis in the class makes no mistakes under the true data distribution.

7.2 Consistency of ERM

If $L_{\mathcal{D}}(h^*) = 0$, then with probability one h^* makes no mistakes on the training sample. Therefore

$$L_S(h^*) = 0.$$

Since $\hat{h}$ is an empirical risk minimizer,

$$L_S(\hat{h}) \leq L_S(h^*) = 0.$$

Empirical risk is nonnegative, so

$$L_S(\hat{h}) = 0.$$

Thus, in the realizable setting, ERM returns a hypothesis that is consistent with the sample.

7.3 Bad Hypotheses

Fix an accuracy parameter $\epsilon > 0$. Define the set of bad hypotheses:

$$B = \{h \in \mathcal{H} : L_{\mathcal{D}}(h) > \epsilon\}.$$

If ERM fails to return a good hypothesis, then $L_{\mathcal{D}}(\hat{h}) > \epsilon$. But in the realizable setting ERM is consistent, so this means some bad hypothesis must have empirical risk zero. Therefore

$$\{L_{\mathcal{D}}(\hat{h}) > \epsilon\} \subseteq \{\exists h \in B : L_S(h) = 0\}.$$

Taking probabilities gives

$$\Pr[L_{\mathcal{D}}(\hat{h}) > \epsilon] \leq \Pr[\exists h \in B : L_S(h) = 0].$$

7.4 One Bad Hypothesis

Fix $h \in B$. Since h is bad,

$$L_{\mathcal{D}}(h) > \epsilon.$$

This means the probability that h makes a mistake on one fresh sample is more than ϵ . Therefore the probability that h is correct on one sample is less than $1 - \epsilon$.

For h to have zero empirical risk, it must be correct on all m independent examples. Hence

$$\Pr[L_S(h) = 0] = (1 - L_{\mathcal{D}}(h))^m \leq (1 - \epsilon)^m.$$

Using the standard inequality $1 - \epsilon \leq e^{-\epsilon}$, we obtain

$$\Pr[L_S(h) = 0] \leq e^{-\epsilon m}.$$

7.5 Union Bound

Using the union bound over all bad hypotheses,

$$\begin{aligned} \Pr[\exists h \in B : L_S(h) = 0] &\leq \sum_{h \in B} \Pr[L_S(h) = 0] \\ &\leq |B|e^{-\epsilon m} \\ &\leq |\mathcal{H}|e^{-\epsilon m}. \end{aligned}$$

Thus

$$\Pr[L_{\mathcal{D}}(\hat{h}) > \epsilon] \leq |\mathcal{H}|e^{-\epsilon m}.$$

7.6 Sample Complexity

To make the failure probability at most δ , it is enough to require

$$|\mathcal{H}|e^{-\epsilon m} \leq \delta.$$

Taking logarithms:

$$\log |\mathcal{H}| - \epsilon m \leq \log \delta.$$

Rearranging:

$$\epsilon m \geq \log |\mathcal{H}| + \log \frac{1}{\delta}.$$

Therefore it is enough that

$$m \geq \frac{1}{\epsilon} \left(\log |\mathcal{H}| + \log \frac{1}{\delta} \right).$$

Theorem 1 (Realizable finite-class guarantee, informal). If $\mathcal{H}$ is finite, the setting is realizable, and

$$m \geq \frac{1}{\epsilon} \left(\log |\mathcal{H}| + \log \frac{1}{\delta} \right),$$

then with probability at least $1 - \delta$, every hypothesis in $\mathcal{H}$ that is consistent with the sample has true risk at most ϵ .

7.7 Lean Status

In Lean, the probability-heavy realizable concentration theorem is stated as an explicit assumption:

`finiteClass_realizable_concentration.`

The theorem wrapper is:

`realizable_learning_guarantee.`

It is located in:

`LearningTheoryProject/Realizable.lean.`

The Lean theorem does not reprove Hoeffding, the union bound, or the logarithmic sample complexity algebra. Instead, it formalizes the surrounding theorem shape using a named concentration assumption.

8 Agnostic Setting

8.1 Assumption Removed

The agnostic setting removes the assumption that a perfect hypothesis exists. There may be no $h \in \mathcal{H}$ with $L_{\mathcal{D}}(h) = 0$.

Instead, define h^* to be a best hypothesis in $\mathcal{H}$:

$$h^* \in \arg \min_{h \in \mathcal{H}} L_{\mathcal{D}}(h).$$

The goal is not perfection. The goal is to show that ERM performs almost as well as h^* .

8.2 Uniform Convergence

The key deterministic assumption is uniform convergence:

$$\forall h \in \mathcal{H}, \quad |L_{\mathcal{D}}(h) - L_S(h)| \leq \epsilon.$$

This absolute-value statement gives two directed inequalities:

$$L_{\mathcal{D}}(h) \leq L_S(h) + \epsilon,$$

and

$$L_S(h) \leq L_{\mathcal{D}}(h) + \epsilon.$$

Lean represents this directly as two inequalities rather than as an absolute value. This is one example of Lean forcing a paper proof to become more explicit.

8.3 Deterministic Agnostic ERM Lemma

Lemma 1 (Deterministic agnostic ERM guarantee). Assume uniform convergence and assume $\hat{h}$ is an empirical risk minimizer. Then for any comparator h^* ,

$$L_{\mathcal{D}}(\hat{h}) \leq L_{\mathcal{D}}(h^*) + 2\epsilon.$$

Proof. Start with the true risk of $\hat{h}$. By uniform convergence applied to $\hat{h}$,

$$L_{\mathcal{D}}(\hat{h}) \leq L_S(\hat{h}) + \epsilon.$$

Since $\hat{h}$ is an empirical risk minimizer,

$$L_S(\hat{h}) \leq L_S(h^*).$$

Adding ϵ to both sides gives

$$L_S(\hat{h}) + \epsilon \leq L_S(h^*) + \epsilon.$$

By uniform convergence applied to h^* in the other direction,

$$L_S(h^*) \leq L_{\mathcal{D}}(h^*) + \epsilon.$$

Adding ϵ gives

$$L_S(h^*) + \epsilon \leq L_{\mathcal{D}}(h^*) + \epsilon + \epsilon.$$

Combining the inequalities:

$$\begin{aligned} L_{\mathcal{D}}(\hat{h}) &\leq L_S(\hat{h}) + \epsilon \\ &\leq L_S(h^*) + \epsilon \\ &\leq L_{\mathcal{D}}(h^*) + \epsilon + \epsilon \\ &= L_{\mathcal{D}}(h^*) + 2\epsilon. \end{aligned}$$

Thus ERM is within 2ϵ of the comparator hypothesis. □

The two ϵ terms come from applying uniform convergence twice: once for $\hat{h}$ and once for h^* .

8.4 Lean Status

This deterministic result is the strongest fully verified theorem in the project. In Lean it is named:

`agnostic_erm_deterministic.`

It is located in:

`LearningTheoryProject/Agnostic.lean.`

The Lean proof follows the same chain:

- apply uniform convergence to `hHat`,
- use the ERM inequality comparing `hHat` to `hStar`,
- add `eps` to both sides with `Nat.add_le_add_right`,
- apply uniform convergence to `hStar`,
- rewrite associativity with `Nat.add_assoc`.

8.5 High-Probability Agnostic Bound

For a fixed hypothesis h , empirical risk is an average of independent zero-one random variables:

$$\mathbf{1}[h(x_i) \neq y_i].$$

Its expectation is the true risk:

$$\mathbb{E}[L_S(h)] = L_{\mathcal{D}}(h).$$

Hoeffding's inequality gives

$$\Pr[|L_S(h) - L_{\mathcal{D}}(h)| > \epsilon] \leq 2e^{-2m\epsilon^2}.$$

Using the union bound over $\mathcal{H}$:

$$\begin{aligned} \Pr[\exists h \in \mathcal{H} : |L_S(h) - L_{\mathcal{D}}(h)| > \epsilon] &\leq \sum_{h \in \mathcal{H}} \Pr[|L_S(h) - L_{\mathcal{D}}(h)| > \epsilon] \\ &\leq 2|\mathcal{H}|e^{-2m\epsilon^2}. \end{aligned}$$

To make this failure probability at most δ , require

$$2|\mathcal{H}|e^{-2m\epsilon^2} \leq \delta.$$

Solving gives

$$m \geq \frac{1}{2\epsilon^2} \log \frac{2|\mathcal{H}|}{\delta}.$$

Therefore, with probability at least $1 - \delta$,

$$L_{\mathcal{D}}(\hat{h}) \leq \min_{h \in \mathcal{H}} L_{\mathcal{D}}(h) + 2\epsilon.$$

In Lean, the finite-class uniform convergence theorem is assumed as:

`finiteClass_agnostic_uniform_convergence.`

The high-probability wrapper theorem is:

`agnostic_high_probability_guarantee.`

9 Why the Rates Differ

The realizable rate is:

$$m = O\left(\frac{\log|\mathcal{H}| + \log(1/\delta)}{\epsilon}\right).$$

The agnostic rate is:

$$m = O\left(\frac{\log|\mathcal{H}| + \log(1/\delta)}{\epsilon^2}\right).$$

The difference comes from the type of event being bounded.

In the realizable setting, the proof asks whether a bad hypothesis makes zero mistakes on all samples. That probability decays like

$$e^{-m\epsilon}.$$

In the agnostic setting, the proof asks whether empirical averages are close to expectations for all hypotheses. Hoeffding's inequality gives decay like

$$e^{-m\epsilon^2}.$$

Solving these exponential bounds for m gives the different rates.

10 Lean Formalization

10.1 Design Choice: Abstract Risk

The Lean project uses:

```
abbrev Risk := Nat.
```

This is an abstraction. It does not claim that risks are literally natural numbers. The purpose is to focus on the deterministic order reasoning needed for the ERM theorem while avoiding a full formalization of real-valued probability.

10.2 Main Definitions

The core structure is:

```
LearningProblem Hypothesis.
```

It contains:

- `trueRisk : Hypothesis -> Risk`,
- `empiricalRisk : Hypothesis -> Risk`,
- `classMember : Hypothesis -> Prop`,
- `classSize : Nat`.

The main predicates are:

- `InClass`,
- `NonemptyClass`,
- `Consistent`,
- `BadHypothesis`,
- `Realizable`,
- `UniformConvergence`,
- `EmpiricalRiskMinimizer`,
- `TrueRiskOptimal`.

10.3 Lean Theorem Interfaces and Assumptions

The Lean project distinguishes ordinary definitions, proved deterministic lemmas, and assumed probability interfaces.

Definitions. The following objects are definitions, not assumptions:

- `Consistent P h`: the empirical risk of `h` is zero.
- `BadHypothesis P eps h`: the true risk of `h` is greater than `eps`.
- `Realizable P`: there exists a class member with zero true risk.
- `UniformConvergence P eps`: both directed risk inequalities hold for every hypothesis.
- `EmpiricalRiskMinimizer P hHat`: `hHat` has empirical risk no larger than any comparator.
- `TrueRiskOptimal P hStar`: `hStar` has true risk no larger than any comparator.

Proved deterministic helper lemmas. The following are proved directly from definitions:

- `uniform_true_le_empirical_plus`,
- `uniform_empirical_le_true_plus`,
- `consistent_empirical_zero`,
- `class_member_iff_inClass`.

Assumed probability interfaces. The following are axioms in `AssumedResults.lean`:

- `HighProbabilityEvent`: represents an event holding with probability at least $1 - \delta$.
- `highProbability_mono`: if event p implies event q , then a high-probability guarantee for p gives a high-probability guarantee for q .
- `finiteClass_realizable_concentration`: the assumed finite-class realizable concentration theorem.
- `finiteClass_agnostic_uniform_convergence`: the assumed finite-class agnostic uniform convergence theorem.

Main proved theorem. The theorem `agnostic_erm_deterministic` is fully proved in Lean. It uses no probability axiom directly; it uses only uniform convergence, ERM, and elementary natural-number order reasoning.

Wrapper theorems. The theorems `realizable_learning_guarantee` and `agnostic_high_probability_guarantee` compile in Lean but depend on the assumed concentration interfaces. They are included to show the final learning-theory theorem shape while keeping the probability boundary explicit.

Component	Status
-----------	--------

10.4 Formalization Boundary

The project separates proved results from assumed results.

Component	Status
Basic learning definitions	Formalized in Lean.
Consistency, realizability, ERM, uniform convergence	Formalized in Lean.
Deterministic agnostic ERM theorem	Fully proved in Lean.
Agnostic high-probability wrapper	Proved using an assumed concentration theorem.
Realizable learning guarantee wrapper	Proved using an assumed concentration theorem.
Hoeffding inequality	Assumed/scoped out.
Finite-class probability union bound	Assumed/scoped out.
Exact logarithmic sample complexity in Lean	Explained mathematically, not fully formalized.
Full MA-LoT system	Not implemented; workflow is MA-LoT-inspired.

11 Repository Layout

The repository is organized so that the Lean code, mathematical notes, presentation materials, AI workflow artifacts, and final report are separated.

11.1 Lean Files

File	Purpose
LearningTheoryProject/BasicDefinitions.lean	Defines the abstract learning vocabulary: risk, classifiers, learning problems, class membership, consistency, realizability, ERM, and uniform convergence.
LearningTheoryProject/AssumedResults.lean	States the high-probability interface and assumed concentration results.
LearningTheoryProject/Realizable.lean	Contains helper lemmas and the realizable theorem wrapper.
LearningTheoryProject/Agnostic.lean	Contains the fully proved deterministic agnostic ERM theorem and the high-probability agnostic wrapper.
LearningTheoryProject/DemoExamples.lean	Contains small examples used in the live presentation.

File	Purpose
LearningTheoryProject/MainTheorem.lean	Imports the complete formalization and describes the main results.

11.2 Notes and Proof Explanation

- `notes/math_overview.md`: overview of the learning problem.
- `notes/proof_realizable.md`: realizable proof explanation.
- `notes/proof_agnostic.md`: agnostic proof explanation.
- `notes/lean_translation_notes.md`: paper proof to Lean translation notes.

11.3 Presentation Materials

- `presentation/final_slides.pptx`: final delivered slide deck.
- `presentation/speaker_notes.md`: slide-by-slide speaking script.
- `presentation/annotated_proof_walkthrough.md`: Lean proof next to human explanation.
- `presentation/paper_vs_lean_table.md`: informal proof phrases versus explicit Lean requirements.
- `presentation/proof_dependency_graph.md`: dependency diagram for definitions, assumptions, and theorems.
- `presentation/proof_pipeline_diagram.md`: proof generation and repair workflow.
- `presentation/visual_theorem_flow.md`: theorem-flow explanation.

12 MA-LoT-Inspired Proof Repair Workflow

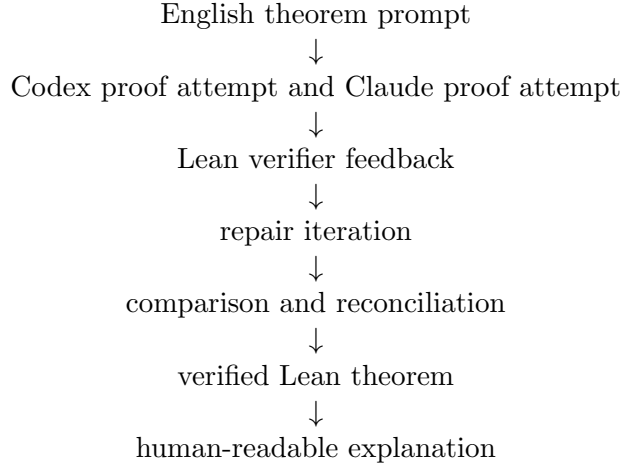
12.1 What MA-LoT Contributes Conceptually

MA-LoT motivates a theorem-proving workflow where natural-language reasoning, Lean verification, and proof repair interact. The important lesson for this project is that an LLM-generated proof is not trusted directly. It is checked by Lean, and Lean’s feedback is used to guide correction.

This project is explicitly MA-LoT-inspired. It does not implement the full MA-LoT system.

12.2 Adapted Workflow

The adapted workflow is:



12.3 Why This Matters

LLMs can generate plausible-looking proofs that are not correct Lean proofs. Lean catches errors such as:

- missing assumptions,
- wrong inequality directions,
- hallucinated lemma names,
- arithmetic gaps,
- tactic failures,
- theorem statements that are too informal.

The workflow turns those errors into repair data.

12.4 AI Collaboration Reflection

Across the project, AI assistance was most useful for proposing proof structure, drafting explanatory text, and producing alternative Lean attempts that could be checked against the verifier. The collaboration evolved in three stages.

Planning stage. AI assistance helped turn the broad project idea into a scoped plan: formalize finite-class learning, prioritize the deterministic agnostic theorem, isolate probability facts as assumptions, and build explanation artifacts in parallel with Lean code.

Proof stage. AI-generated proof attempts were useful as drafts, but not as final authority. The most important rule was that no proof was treated as correct until Lean accepted it. The AI was delegated proof-attempt generation, proof repair suggestions, and explanation drafting. Lean was the verifier.

Presentation stage. AI assistance helped convert formal Lean statements into human-readable proof walkthroughs, diagrams, speaker notes, and the final report. This was especially useful for explaining what Lean forced to become explicit: directed inequalities, named assumptions, and arithmetic rewrites.

The workflow also showed where AI can mislead. It sometimes used theorem names that did not exist, treated a conjunction as if it were a single inequality, tried automation that could not see the right constraints, or skipped arithmetic steps that a paper proof would call “obvious.” These failures became useful only after Lean exposed them and they were logged, diagnosed, and repaired.

13 Multi-Model Comparison Workflow

13.1 Blind Generation

Codex and Claude are given the same theorem prompt:

```
theorem_prompts/agnostic_erm_theorem.md.
```

The final verified Lean proof is not given to them during blind generation. This prevents the models from merely copying the final proof and makes the comparison more meaningful.

13.2 Generated Proof Artifacts

The generated files are stored in:

- `generated_proofs/codex/agnostic_attempt_1.lean`,
- `generated_proofs/codex/agnostic_attempt_2_repaired.lean`,
- `generated_proofs/codex/simplified_attempt.lean`,
- `generated_proofs/claude/agnostic_attempt_1.lean`,
- `generated_proofs/claude/agnostic_attempt_2_repaired.lean`,
- `generated_proofs/claude/simplified_attempt.lean`.

13.3 Comparison Script

The script

```
multi_model_workflow/run_comparison.py
```

checks each generated file with Lean and records structural features.

It compares:

- whether the file compiles,
- Lean’s exit code,
- verifier error output,
- line count,
- SHA hash prefix,

- whether the proof uses `calc`,
- whether the proof uses `omega`,
- whether it uses `Nat.add_le_add_right`,
- whether it uses `Nat.add_assoc`,
- whether it calls the main theorem directly,
- whether it extracts conjunction directions using `.left` or `.right`.

The comparison is not a deep semantic equivalence checker. Lean verifies whether each proof checks against its theorem statement. The script then compares proof structure and usefulness for explanation or simplification.

13.4 Repair and Simplification

The comparison script generates:

- `multi_model_workflow/comparison_report.md`,
- `multi_model_workflow/repair_recommendations_for_models.md`.

The repair recommendations tell Codex and Claude what to borrow from each other. For example:

- Codex should explicitly extract uniform convergence directions.
- Claude should avoid relying on `omega` for abstract `Nat`-valued risk functions.
- Both should preserve a readable `calc` chain for explanation.

Simplification is only accepted after Lean verification. A shorter proof is not kept merely because it looks cleaner; it must still pass `lake build`.

13.5 How Comparison Prevents Shortcuts

The comparison workflow prevents shortcut behavior in three ways.

1. Both models start from the same theorem prompt, not from the final proof.
2. Lean checks whether each generated proof actually compiles.
3. The comparison exposes missing assumptions, hallucinated lemmas, tactic failures, and proofs that only sound correct in English.

Thus, the LLM is a proof assistant, but Lean remains the verifier.

14 Proof Repair Case Study

The agnostic ERM theorem was the main repair case study.

14.1 Failure Pattern 1: Uniform Convergence Treated Too Informally

Informally, the proof says “by uniform convergence.” In Lean, uniform convergence is a conjunction with two directed inequalities. A failed proof attempt tried to use the whole uniform convergence hypothesis directly.

The repair was to extract the needed direction:

`(hUC hHat).left`

or

`(hUC hStar).right.`

14.2 Failure Pattern 2: ERM Used Before Matching the Goal

ERM gives:

$$L_S(\hat{h}) \leq L_S(h^*).$$

But the proof goal needs:

$$L_S(\hat{h}) + \epsilon \leq L_S(h^*) + \epsilon.$$

The repair was:

`Nat.add_le_add_right hERMStar eps.`

14.3 Failure Pattern 3: Arithmetic Hidden by “Rearranging”

The paper proof says the result follows by rearranging. Lean requires the exact final expression:

`P.trueRisk hStar + (eps + eps).`

The repair used:

`rw [Nat.add_assoc].`

15 Verification and Reproducibility

The project was verified with Lean 4.29.1 and Lake 5.0.0.

15.1 Build Command

To verify the Lean project:

`lake build`

Expected output:

Build completed successfully (16 jobs).

15.2 Comparison Command

To reproduce the multi-model comparison:

`python multi_model_workflow/run_comparison.py`

Expected output:

Wrote multi_model_workflow\comparison_report.md

Wrote multi_model_workflow\repair_recommendations_for_models.md

16 Limitations

- Risks are abstract `Nat` values, not real-valued probabilities.
- Hoeffding’s inequality is not formalized in Lean.
- The finite-class probability union bound is not formalized in Lean.
- Exact logarithmic sample complexity bounds are proved in the mathematical explanation but represented abstractly in Lean.
- The project is MA-LoT-inspired but does not implement the full MA-LoT framework.
- The comparison script performs verification and structural comparison, not general semantic equivalence checking between arbitrary Lean files.

17 Future Work

Future extensions could:

- replace assumed concentration facts with Mathlib probability theorems,
- use real-valued risks,
- formalize Hoeffding’s inequality,
- formalize exact sample complexity algebra in Lean,
- automate more of the MA-LoT-style repair loop,
- integrate an MA-LoT-related model such as a LoT-Solver model as a proof generation or correction agent,
- evaluate more theorem prompts with the same comparison pipeline.

18 Conclusion

This project formalizes the deterministic core of finite-class learning guarantees in Lean 4 and makes the probability boundary explicit. The deterministic agnostic ERM theorem is fully verified. The realizable and agnostic high-probability results are represented using clearly named assumed concentration results.

The project also contributes an MA-LoT-inspired proof repair study. Codex and Claude produce proof attempts, Lean verifies them, errors are logged, repairs are made, attempts are compared, and the final verified proof is explained for a human reader.

The final result is both a Lean formalization and a small study of AI-assisted theorem proving, proof repair, and proof explainability.

A Appendix A: File Index

A.1 Core Lean Formalization

- `LearningTheoryProject/BasicDefinitions.lean`
- `LearningTheoryProject/AssumedResults.lean`
- `LearningTheoryProject/Realizable.lean`
- `LearningTheoryProject/Agnostic.lean`
- `LearningTheoryProject/DemoExamples.lean`
- `LearningTheoryProject/MainTheorem.lean`

A.2 Mathematical Notes

- `notes/math_overview.md`
- `notes/proof_realizable.md`
- `notes/proof_agnostic.md`
- `notes/lean_translation_notes.md`

A.3 MA-LoT-Inspired Workflow

- `malot_workflow/README.md`
- `malot_workflow/malot_overview.md`
- `malot_workflow/proof_attempt_log.md`
- `malot_workflow/verifier_feedback_log.md`
- `malot_workflow/repair_iterations.md`
- `malot_workflow/success_failure_table.md`
- `malot_workflow/pattern_analysis.md`
- `malot_workflow/agnostic_repair_case_study.md`
- `malot_workflow/final_reflection.md`

A.4 Multi-Model Workflow

- `theorem_prompts/agnostic_erm_theorem.md`
- `theorem_prompts/realizable_theorem.md`
- `theorem_prompts/simplification_prompt.md`
- `generated_proofs/codex/`

- generated_proofs/claude/
- multi_model_workflow/run_comparison.py
- multi_model_workflow/comparison_report.md
- multi_model_workflow/repair_recommendations_for_models.md
- multi_model_workflow/codex_vs_claude_comparison.md
- multi_model_workflow/consensus_pipeline.md
- multi_model_workflow/reconciliation_notes.md
- multi_model_workflow/semantic_difference_cases.md

A.5 Proof Refactoring

- proof_refactoring/bloated_vs_simplified.md
- proof_refactoring/verified_refactors.md
- proof_refactoring/failed_simplifications.md

A.6 Presentation and Explainability

- presentation/final_slides.pptx
- presentation/speaker_notes.md
- presentation/annotated_proof_walkthrough.md
- presentation/paper_vs_lean_table.md
- presentation/proof_dependency_graph.md
- presentation/proof_pipeline_diagram.md
- presentation/visual_theorem_flow.md
- presentation/final_demo_examples.md
- presentation/codex_vs_claude_table.md

A.7 Final Submission Files

- README.md
- final_report/final_writeup.tex
- final_report/final_writeup.md
- final_report/submission_checklist.md

B Appendix B: Main Lean Theorem Names

- `uniform_true_le_empirical_plus`
- `uniform_empirical_le_true_plus`
- `realizable_learning_guarantee`
- `agnostic_erm_deterministic`
- `agnostic_erm_against_optimal`
- `agnostic_high_probability_guarantee`

C Appendix C: Final Submission Checklist

Before submission:

1. Run `lake build`.
2. Run `python multi_model_workflow/run_comparison.py`.
3. Confirm `presentation/final_slides.pptx` is present.
4. Confirm `final_report/final_writeup.tex` is present.
5. Compile `final_writeup.tex` to PDF if the submission requires PDF.
6. Confirm no PowerPoint lock files are included. These usually begin with a tilde-dollar prefix.
7. Commit or upload all final artifacts.